

Especificaciones de Casos de Prueba - Iteración Refactorizada

Alcance Estricto: Tolerancia a Fallos y Capacidad de Recuperación (ISO/IEC 25010) **Formato:** ISO/IEC/IEEE 29119-3

ID del Caso: CP-001

Nivel y Enfoque: Sistema / Caja Negra **Combinación:** Tolerancia a fallos + Análisis de Valores Límite **Objetivo:** DESTRUCTIVO. Violar el límite de la regla de negocio creando el tercer préstamo activo en simultáneo para corromper el validador. **Precondiciones:** El usuario existe en BD y tiene exactamente 2 préstamos activos (límite máximo).

Datos de Entrada: `correoUsuario = "tester@ucab.edu"`, `isbn = 9781234567897`, `fechaPrestamoStr = "2026-07-12"` **Pasos de**

Ejecución:

1. Autenticarse con rol USUARIO y obtener token.
2. Interceptar la petición de red.
3. Enviar un POST a `/api/prestar` con el payload de entrada.
4. Repetir la misma petición POST en menos de 50ms usando hilos concurrentes para provocar una condición de carrera.

Resultados Esperados: El sistema DEBE rechazar la petición con HTTP 400 y el mensaje "El usuario ya tiene 2 préstamos activos." SIN crear el tercer préstamo. **Criterios de Aceptación:** El registro en BD muestra

`countByUsuarioIdAndFechaDevolucionIsNull = 2.`

Trazabilidad: `PrestamoService.crearPrestamo()` (líneas 36-37) y `Controller.registrarPrestamo()`.

ID del Caso: CP-002 (Nuevo - Frontend E2E)

Nivel y Enfoque: Sistema / Caja Negra (E2E Frontend) **Combinación:** Tolerancia a Fallos + Pruebas de Transición de Estados **Objetivo:** DESTRUCTIVO. Provocar una falla de red durante la carga de amonestaciones para evidenciar el colapso de la UI por promesas no manejadas. **Precondiciones:** Frontend Vue.js ejecutándose. Usuario autenticado navegando hacia la vista `VerificarPago.vue`. Conexión de red del navegador estrangulada (Offline). **Datos de Entrada:** Evento de montaje del componente (`onMounted` llama a `cargarAmonestaciones`). **Pasos de Ejecución:**

1. Desactivar la conexión a red en las herramientas de desarrollo del navegador.
2. Acceder a la ruta correspondiente a `VerificarPago.vue`.
3. Observar la renderización de la pantalla y la consola de errores.

Resultados Esperados: La prueba VA A FALLAR exponiendo los defectos WT-03 y WT-06. La promesa de Axios será rechazada, pero al carecer de bloque `try/catch` y de interceptor global, el frontend colapsará visualmente. Se registrará un *unhandled promise rejection* en consola y no se mostrará ninguna pantalla de fallback ni toast informativo. **Criterios de Aceptación:** Se documenta el quiebre del estado de la UI y la exposición del error técnico en consola sin manejo elegante. **Trazabilidad:** `VerificarPago.vue` -> `cargarAmonestaciones()`, falta de interceptor en `main.js`.

ID del Caso: CP-003 (Refactorizado - Backend Fechas)

Nivel y Enfoque: Sistema / Caja Negra **Combinación:** Tolerancia a Fallos + Partición de Equivalencia **Objetivo:** DESTRUCTIVO. Inyectar datos en una partición inválida anómala para tumbar el endpoint mediante un fallo de parseo. **Precondiciones:** API desplegada y accesible. **Datos de Entrada:** Payload de préstamo con `fechaPrestamoStr = "9999-99-99"` (Formato de fecha inválido). **Pasos de Ejecución:**

1. Construir un JSON de creación de préstamo con la fecha irreal.

2. Enviar petición POST a `/api/prestar`. **Resultados**

Esperados: La prueba VA A FALLAR exponiendo el defecto WT-01. Al intentar hacer `LocalDate.parse()` sin una envoltura `try/catch` ni un manejador global (WT-02), el sistema gatillará un error HTTP 500, devolviendo el `stacktrace` completo de `DateTimeParseException` al cliente. **Criterios de**

Aceptación: Se registra el HTTP 500 y la filtración del `stacktrace`. El tester reporta la nula tolerancia a fallos en el parsing.

Trazabilidad: `PrestamoService.crearPrestamo()` (línea 47).

ID del Caso: CP-004

Nivel y Enfoque: Integración / Caja Gris **Combinación:** Tolerancia a fallos + Adivinación de Errores (Error Guessing) **Objetivo:** DESTRUCTIVO. Explotar la lógica de devolución para incrementar falsamente el inventario de libros (Ataque de Doble Gasto).

Precondiciones: Préstamo ID 5 activo. Libro ID 10 con `cantidad = 5`.

Datos de Entrada: `prestamoId = 5`. **Pasos de Ejecución:**

1. Abrir dos terminales paralelas (o usar JMeter).
 2. Ejecutar simultáneamente `POST /api/prestamos/devolver?prestamoId=5` en ambas terminales. **Resultados Esperados:** La primera petición retorna "Préstamo devuelto con éxito." (cantidad a 6). La segunda DEBE retornar "El préstamo ya fue devuelto." y NO aumentar la cantidad a 7. **Criterios de Aceptación:** Al consultar la BD, el libro ID 10 tiene `cantidad = 6`, y no hay inconsistencias. **Trazabilidad:** `PrestamoService.devolverPrestamo()` (líneas 72-74 y 81-83).
-

ID del Caso: CP-005

Nivel y Enfoque: Integración / Caja Blanca **Combinación:** Capacidad de Recuperación + Análisis de Flujo de Datos **Objetivo:** DESTRUCTIVO. Romper el flujo de la transacción a la mitad para

validar si el sistema deja datos corruptos o huérfanos. **Precondiciones:** Contexto de Spring cargado. `prestamoRepository` funcional, `libroRepository` forzado a arrojar `DataAccessException`. **Datos de Entrada:** Petición válida para crear préstamo. **Pasos de Ejecución:**

1. Llamar a `PrestamoService.crearPrestamo`.
2. La línea 58 (`prestamoRepository.save`) guarda el préstamo en BD.
3. La línea 61 (`libroRepository.save`) arroja la excepción programada. **Resultados Esperados:** La prueba VA A FALLAR exponiendo el defecto arquitectónico WT-04. Al carecer de `@Transactional`, el sistema no tiene capacidad de rollback. El préstamo queda guardado, pero el libro no se descuenta.

Criterios de Aceptación: Se reporta el hallazgo crítico exigiendo refactorización y control transaccional. **Trazabilidad:**

`PrestamoService.crearPrestamo()` (líneas 58 a 61).

ID del Caso: CP-006

Nivel y Enfoque: Integración / Caja Negra **Combinación:** Tolerancia a fallos + Transición de Estados **Objetivo:** DESTRUCTIVO. Forzar una transición de estado ilegal en la máquina de estados del préstamo.

Precondiciones: Préstamo ID 8 en estado "activo". Autenticado como "BIBLIOTECARIO". **Datos de Entrada:** Petición de renovación sobre `id` = 8. **Pasos de Ejecución:**

1. Validar en BD que el préstamo está activo (`fechaDevolucion` es NULL).
2. Enviar petición PUT directa a `/api/prestamos/8/renovar`.

Resultados Esperados: El sistema debe impedir la transición saltándose el estado "finalizado", retornando HTTP 400. **Criterios de Aceptación:** El estado original no se altera. **Trazabilidad:**

`PrestamoService.renovarPrestamo()` (líneas 132-134).

ID del Caso: CP-007 (Nuevo - Frontend E2E)

Nivel y Enfoque: Sistema / Caja Negra (E2E Frontend) **Combinación:** Capacidad de Recuperación + Pruebas de Escenario **Objetivo:** DESTRUCTIVO. Simular una falla del backend al agregar una reseña para verificar la degradación elegante de la interfaz. **Precondiciones:** Usuario en `PantallaLibro.vue`. Interceptor de red mockeado para devolver HTTP 500 en el POST de reseñas. **Datos de Entrada:** Formulario de reseña válido y pulsación en "Agregar Reseña". **Pasos de Ejecución:**

1. Llenar el formulario de reseña.
 2. Clic en "Agregar Reseña", ejecutando `agregarResena`.
 3. El backend simulado retorna HTTP 500. **Resultados Esperados:** La prueba VA A FALLAR exponiendo WT-03 y WT-06. Como `agregarResena` carece de bloque `catch`, el error escapa. La UI no se recupera, dejando el formulario sin limpiar y arrojando un *unhandled promise rejection* en consola, frustrando la interacción. **Criterios de Aceptación:** Documentación de la nula resiliencia de la UI ante excepciones de escritura. **Trazabilidad:** `PantallaLibro.vue -> agregarResena()`.
-

ID del Caso: CP-008

Nivel y Enfoque: Aceptación / Caja Negra **Combinación:** Capacidad de Recuperación + Pruebas Aleatorias (Fuzzing) **Objetivo:** DESTRUCTIVO. Ahogar el parseo JSON del backend inyectando payloads gigantescos. **Precondiciones:** API ejecutándose. **Datos de Entrada:** Petición con cuerpo JSON mutado: `{ "texto": "A" * 10000000 } (10MB)`. **Pasos de Ejecución:**

1. Generar el payload inmenso.
 2. Enviar PUT a `/api/resenas/1`. **Resultados Esperados:** El servidor web intercepta la petición (`Payload Too Large`), recuperándose sin colapsar el servicio. **Criterios de Aceptación:** Retorna HTTP 413, normalizando el consumo de RAM. **Trazabilidad:** `Controller.editarResena()` (línea 258).
-

ID del Caso: CP-009 (Nuevo - Frontend E2E)

Nivel y Enfoque: Sistema / Caja Negra (E2E Frontend) **Combinación:** Tolerancia a Fallos + Análisis de Valores Límite **Objetivo:** DESTRUCTIVO. Interrumpir el flujo de pago con un error 502 para evidenciar fallas silenciosas en la mutación asíncrona. **Precondiciones:** Usuario en `SolicitudVerificacionPago.vue` con amonestación lista. API mockeada para retornar 502 Bad Gateway. **Datos de Entrada:** Evento de clic en confirmación de pago. **Pasos de Ejecución:**

1. Clic en "Pagar amonestación".
 2. El servidor responde HTTP 502. **Resultados Esperados:** La prueba VA A FALLAR por WT-03 y WT-06. La función `pagarAmonestacion` utiliza `try/finally` pero omite `catch`. El sistema apaga el spinner pero no muestra ningún `toast` o `fallback`, dejando al usuario bloqueado y con el error en consola. **Criterios de Aceptación:** Constatación del escape de la excepción y falta de feedback al usuario. **Trazabilidad:** `SolicitudVerificacionPago.vue -> pagarAmonestacion()`.
-

ID del Caso: CP-010

Nivel y Enfoque: Sistema / Caja Gris **Combinación:** Tolerancia a fallos + Pruebas de Escenario **Objetivo:** DESTRUCTIVO. Eliminar un perfil de usuario con dependencias duras activas para corromper la integridad referencial. **Precondiciones:** Usuario "MOROSO" con 1 préstamo sin devolver. **Datos de Entrada:** Petición DELETE autenticada por "MOROSO". **Pasos de Ejecución:**

1. Iniciar sesión como "MOROSO".
2. Enviar DELETE a `/api/usuarios`. **Resultados Esperados:** La prueba VA A FALLAR exponiendo WT-02. Al no existir `@RestControllerAdvice`, la violación de la *foreign key constraint* en la BD generará un HTTP 500 incontrolado que expondrá detalles internos de SQL al cliente. **Criterios de Aceptación:** Se evidencia el HTTP 500 y la filtración de la excepción. **Trazabilidad:** `Controller.eliminarPerfil()`

ID del Caso: CP-011 (Modificado - Control de Timeout)

Nivel y Enfoque: Integración / Caja Gris **Combinación:** Tolerancia a fallos + Adivinación de Errores **Objetivo:** Someter al sistema a un fallo de latencia extrema en BD para verificar el corte de peticiones encoladas. **Precondiciones:** `amonestacionRepository` detrás de Toxiproxy con latencia de 30s. **Datos de Entrada:** Petición POST válida a `/api/prestamos/devolver`. **Pasos de Ejecución:**

1. Configurar proxy para demorar 30 segundos.
2. Ejecutar petición HTTP. **Resultados Esperados:** Aunque JPA aplique un timeout, al faltar un controlador global (WT-02), el sistema abortará devolviendo un HTTP 500 con el stacktrace completo del timeout, fallando en proveer un 503 controlado. **Criterios de Aceptación:** Se reporta la exposición del stacktrace tras el timeout. **Trazabilidad:**

```
Controller.devolverPrestamo().
```

ID del Caso: CP-012 (Modificado - Fallo Real de BD)

Nivel y Enfoque: Integración / Caja Blanca **Combinación:** Capacidad de Recuperación + Pruebas de Recuperación **Objetivo:** DESTRUCTIVO. Simular caducidad de conexiones para validar el comportamiento real del pool. **Precondiciones:** Aplicación conectada a MySQL, transcurridas >8 horas de inactividad. **Datos de Entrada:** Petición GET a `/api/libros`. **Pasos de Ejecución:**

1. Esperar timeout de inactividad de MySQL (o detenerlo forzosamente y reiniciarlo).
2. Enviar petición a la API. **Resultados Esperados:** La prueba VA A FALLAR confirmando WT-05 y WT-02. Al no configurar `max-lifetime` y `keepalive-time` en HikariCP, el pool intentará usar conexiones muertas. El sistema NO se recuperará y arrojará el error `Communications link failure` dentro de un HTTP

500 expuesto al cliente, dejando la app inoperativa. **Criterios de Aceptación:** Se demuestra la caída total que requiere reinicio manual del backend. **Trazabilidad:** Configuración de HikariCP en `application.properties`.

ID del Caso: CP-013 (Nuevo - Frontend E2E)

Nivel y Enfoque: Sistema / Caja Negra (E2E Frontend) **Combinación:** Capacidad de Recuperación + Pruebas Aleatorias **Objetivo:** DESTRUCTIVO. Inyectar timeouts desde el backend al eliminar entidades para validar la nula resiliencia de la UI. **Precondiciones:** Usuario autenticado en `PantallaLibro.vue`. Interceptor de red forzando HTTP 504. **Datos de Entrada:** Acción de clic en "Eliminar" comentario. **Pasos de Ejecución:**

1. Clic en el botón eliminar.
2. Retener petición 15 segundos hasta el timeout de red.

Resultados Esperados: La prueba VA A FALLAR exponiendo WT-03 y WT-06. Sin bloque `catch` en `eliminarComentarioResena`, el frontend queda pasmado. No hay mensaje de timeout, dejando al usuario congelado mientras la consola acumula errores. **Criterios de Aceptación:** Constatación empírica de la falta de recuperación del frontend ante timeouts.

Trazabilidad: `PantallaLibro.vue` -> `eliminarComentarioResena()`.

ID del Caso: CP-014

Nivel y Enfoque: Sistema / Caja Blanca **Combinación:** Tolerancia a fallos + Análisis de Valores Límite **Objetivo:** DESTRUCTIVO. Forzar un cálculo injusto en penalizaciones por zonas horarias. **Precondiciones:** Préstamo activo al límite de fecha. Servidor en UTC, cliente en UTC-4. **Datos de Entrada:** Devolución en el milisegundo de cambio de día. **Pasos de Ejecución:**

1. Emitir devolución a las 23:59:59 (hora local).

2. Retener y liberar paquete para que el servidor lo procese a las 00:00:01 (UTC). **Resultados Esperados:** Exposición del defecto: `LocalDate.now()` ignora el `ZoneId` y la latencia, imponiendo una amonestación injustificada por el cruce asíncrono. **Criterios de Aceptación:** Se documenta el fallo en la resiliencia temporal y se exige uso de `Instant.now()`. **Trazabilidad:** `PrestamoService.devolverPrestamo()` (líneas 76 y 86-87).